

---

# EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

---

## Operation TELESCREEN - Instructor Solution Key

ARM Cortex-M33 • GDB • Ghidra

September 27, 2026

For educational and defensive security research only

**LEGAL DISCLAIMER:** The information, tools, and code provided in this repository and course are strictly for educational, research, and defensive purposes only.

You are explicitly prohibited from using any materials contained herein to access, test, modify, or exploit any device, network, or system that you do not own 100% or for which you do not have explicit, documented, and legally binding authorization to interact with.

By using this repository and course, you acknowledge and agree that:

1. Any illegal, unauthorized, or malicious use of this information is solely your responsibility.
2. The author(s) and contributor(s) of this repository and course shall not be held liable for any damages, legal repercussions, criminal charges, or unauthorized actions resulting from the use, misuse, or abuse of the contents herein.
3. You will comply with all applicable local, state, national, and international laws regarding cybersecurity and computer fraud.

**IF YOU DO NOT AGREE WITH THESE TERMS, DO NOT USE THIS REPOSITORY AND COURSE.**

---

## Artifact Identity

The instructor-issued artifact hashes are:

```
ctfnode.stripped    af7ab5c2b4837083682db8b54f6892b1a2a33dcc8ce7b202b5a4a5163232da6d
ctfnode.unstripped  6bcee7daa91280eaf02558b1b5f1b5cc8e22a84179605a0725af2b5564257e0a
```

`firmware/ctfnode.unstripped` is the answer key; `firmware/ctfnode.stripped` is the student artifact. Both are built from `ctf/ctfnode.c` by `firmware/build_target.sh` in a pinned `linux/arm64` container.

The correct framing is the **application binary**, not the flash. The four `*.img` files that once appeared in this repository were empty stubs and have been removed; do not expect a boot chain, kernel, device tree, or `mtddparts` anywhere in this challenge.

---

## Task 1: Verify the Artifact (10 points)

### Solution

```
$ shasum -a 256 firmware/ctfnode.stripped
af7ab5c2b4837083682db8b54f6892b1a2a33dcc8ce7b202b5a4a5163232da6d  firmware/ctfnode.stripped

$ file firmware/ctfnode.stripped
ELF 64-bit LSB executable, ARM aarch64, version 1 (SYSV), dynamically linked,
interpreter /lib/ld-linux-aarch64.so.1, for GNU/Linux 3.7.0, stripped
```

ELF identity:

| property | value |
|----------|-------|
| class    | ELF64 |

| property    | value                                  |
|-------------|----------------------------------------|
| data        | little-endian                          |
| machine     | AArch64                                |
| type        | dynamic executable ( EXEC ), PIE-ready |
| interpreter | /lib/ld-linux-aarch64.so.1 (glibc)     |
| symbols     | stripped                               |

Dynamic imports ( `readelf -rW firmware/ctfnodestripped | grep JUMP_SLOT` ):

```
strlen __libc_start_main puts sprintf fclose fopen system
__gmon_start__ abort puts strcmp printf fgets
```

The dangerous import for this binary is `system`. `printf`, `puts`, `putc`, `strlen`, `strcmp`, `sprintf`, `fopen`, `fgets`, `fclose` are ordinary string and I/O helpers; `system` is the one that turns a crafted string into a shell command.

## Grading Rubric (1-to-1 Mapping)

| Criterion                   | Points | Full Credit (Answer Key)                 |
|-----------------------------|--------|------------------------------------------|
| [DOCUMENT] SHA-256 verified | 3      | Matches af7ab5c2...da6d                  |
| [DOCUMENT] ELF identity     | 3      | ELF64, aarch64, dynamic, glibc, stripped |
| [DOCUMENT] Imports flagged  | 4      | All listed; <code>system</code> flagged  |

## Task 2: Recover the Command Surface (15 points)

### Solution

#### Entry chain

```
_start (0x400840)
-> __libc_start_main(main=__wrap_main 0x400874)
   __wrap_main (0x400874): b 0x400800 ; linker --wrap shim
   main (0x400800):      b 0x400c20 ; tail-call
   ctf_dispatch (0x400c20)
```

At `0x40085c - 0x40086c`, `_start` loads `x0 = 0x400874` and calls `__libc_start_main@plt`, so the C entry point glibc invokes is `__wrap_main`, not `main` directly. `__wrap_main` is a one-instruction tail branch to `main`; `main` is a one-instruction tail branch to `ctf_dispatch`.

#### Subcommand table ( `ctf_dispatch`, `0x400c20` )

| subcommand | string address | argc rule               | target         |
|------------|----------------|-------------------------|----------------|
| config     | 0x400ee8       | argc > 2 (needs a path) | ctf_config_run |

| subcommand | string address | argc rule                   | target                              |
|------------|----------------|-----------------------------|-------------------------------------|
| http       | 0x400ef0       | argc > 2 (needs a query)    | ctf_http_handle                     |
| restore    | 0x400ef8       | argc > 2 (needs an archive) | ctf_restore                         |
| login      | 0x400f00       | argc > 3 (user + pass)      | inline strcmp against admin / empty |
| shell      | 0x400f08       | argc > 1 (no argument)      | inline system("/bin/sh")            |
| key        | 0x400f10       | argc > 2 (needs a UID)      | ctf_weak_key + %02x print           |

With `argc < 2`, the dispatcher prints the banner and returns 0. An unknown subcommand also prints the banner and returns 2.

## Grading Rubric (1-to-1 Mapping)

| Criterion                   | Points | Full Credit (Answer Key)                                                          |
|-----------------------------|--------|-----------------------------------------------------------------------------------|
| [DOCUMENT] Entry chain      | 5      | <code>__wrap_main 0x400874 -&gt; main 0x400800 -&gt; ctf_dispatch 0x400c20</code> |
| [DOCUMENT] Subcommand table | 6      | All six strings with addresses                                                    |
| [DOCUMENT] Argument rules   | 4      | <code>argc</code> rules as above                                                  |

## Instructor Notes & Assembly

- Students who stop at `main` and never open `ctf_dispatch` will miss the entire command surface; the interesting code is in the dispatcher.
- The helper functions `ctf_cmd`, `ctf_try_path`, `ctf_try_misc`, and `ctf_print_key` are **inlined** into `ctf_dispatch`, which is why it is the biggest function.

## Task 3: Name Every Function (20 points)

### Solution

The ten application functions, resolved 1:1 against the unstripped twin:

| address  | Ghidra label | resolved name   | rule                               |
|----------|--------------|-----------------|------------------------------------|
| 0x400960 | FUN_00400960 | ctf_crc32_le    | R1; also the 0xEDB88320 polynomial |
| 0x4009b0 | FUN_004009b0 | ctf_weak_key    | R1; 32-iteration loop over the UID |
| 0x400a8c | FUN_00400a8c | ctf_login       | R1; references admin + empty test  |
| 0x400ae0 | FUN_00400ae0 | ctf_config_run  | R1; run= compare + system          |
| 0x400b64 | FUN_00400b64 | ctf_build_cmd   | R1; "ping -c 1 %s"                 |
| 0x400b80 | FUN_00400b80 | ctf_http_handle | R1; system on the built command    |

| address  | Ghidra label | resolved name   | rule                    |
|----------|--------------|-----------------|-------------------------|
| 0x400bc0 | FUN_00400bc0 | ctf_restore     | R1; "tar -xvzf %s -C /" |
| 0x400c00 | FUN_00400c00 | ctf_debug_shell | R1; system("/bin/sh")   |
| 0x400c0c | FUN_00400c0c | ctf_banner      | R1; the banner string   |
| 0x400c20 | FUN_00400c20 | ctf_dispatch    | R1; the strcmp chain    |

Non-application functions resolved by the other rules:

- **R3** PLT lazy resolver at 0x4006f0.
- **R2** the imported glibc thunks at 0x400710 - 0x4007d0 and their .got.plt duplicates at 0x421000 - 0x421070.
- **R4** the phantom FUN\_00400adc at 0x400adc, an alignment-pad twin of ctf\_config\_run.

Inlined away (present in source, absent from the binary):

| helper         | inlined into               |
|----------------|----------------------------|
| ctf_crc32_byte | ctf_crc32_le, ctf_weak_key |
| ctf_cmd        | ctf_dispatch               |
| ctf_print_key  | ctf_dispatch               |
| ctf_try_path   | ctf_dispatch               |
| ctf_try_misc   | ctf_dispatch               |

## Grading Rubric (1-to-1 Mapping)

| Criterion                                        | Points | Full Credit (Answer Key) |
|--------------------------------------------------|--------|--------------------------|
| <b>[DOCUMENT]</b> Ten functions named with rules | 14     | Table above              |
| <b>[DOCUMENT]</b> Inlined helpers                | 6      | All five                 |

## Instructor Notes & Assembly

- Reference output: ghidra/RESOLUTION\_MAP.md, ghidra/resolution.json, and CTF-XX-J-ghidra-function-resolution.md.
- Rebuild with ./firmware/build\_target.sh; open with ./ghidra/make\_project.sh (ghidra/proj/CTFNodeRE.gpr).

## Task 4: Prove the Six Defects (30 points)

### Solution

#### B1 - Config-Sourced Root Execution ( ctf\_config\_run , 0x400ae0 )

```
0x400af0: bl  400760 <fopen@plt>      ; fopen(path, "r")
0x400afc: mov w20, #0x7572
0x400b04: movk w20, #0x3d6e, lsl #16 ; w20 = 0x3d6e7572 = "run=" (LE)
0x400b20: cmp w1, w20                ; is the first 4 bytes "run=" ?
0x400b2c: bl  400770 <system@plt>     ; system(line + 4)
```

Any line in a readable config file beginning `run=` is executed with `system()`. Because the node runs as root, a writable config is a root-execution primitive.

#### B2 - Command Injection ( ctf\_build\_cmd 0x400b64 , ctf\_http\_handle 0x400b80 )

```
0x400b70: adrp x2, 400000
0x400b74: add  x2, x2, #0xe88        ; "ping -c 1 %s"
0x400b78: b    400740 <snprintf@plt>
```

```
0x400b88: adrp x2, 400000
0x400b8c: add  x2, x2, #0xe88        ; "ping -c 1 %s"
0x400ba0: bl   400740 <snprintf@plt>
0x400ba8: bl   400770 <system@plt>   ; system("ping -c 1 <query>")
```

The query is interpolated into a shell command with no quoting, so shell metacharacters (`;`, `|`, ```, `$()`) execute.

#### B3 - Archive-to-Root Restore ( ctf\_restore , 0x400bc0 )

```
0x400bc8: adrp x2, 400000
0x400bcc: add  x2, x2, #0xe98        ; "tar -xvzf %s -C /"
0x400be0: bl   400740 <snprintf@plt>
0x400be8: bl   400770 <system@plt>   ; extracts the upload into /
```

An uploaded archive with `../` entries writes anywhere on the filesystem as root.

#### B4 - Empty / Default Credentials ( ctf\_login , 0x400a8c )

```
0x400ab0: adrp x1, 400000
0x400ab4: add  x1, x1, #0xe68        ; "admin"
0x400ab8: bl   4007b0 <strcmp@plt>
0x400ac0: ldrb w0, [x19]             ; pass[0]
0x400ac4: cmp  w0, #0x0
0x400ac8: cset w20, eq               ; success iff pass[0] == '\0'
```

The user must be `admin` and the password must be the **empty string**. The same logic is duplicated inline in `ctf_dispatch` ( `0x400d0c` - `0x400d40` ).

**B5 - Debug Root Shell ( ctf\_debug\_shell , 0x400c00 )**

```

0x400c00:  adrp x0, 400000
0x400c04:  add x0, x0, #0xeb0 ; "/bin/sh"
0x400c08:  b 400770 <system@plt>

```

The dispatcher reaches an equivalent tail call at 0x400d58 - 0x400d6c .

**B6 - Weak Key Schedule ( ctf\_weak\_key , 0x4009b0 )**

```

0x4009c4:  bl 400710 <strlen@plt> ; ulen = strlen(uid)
0x4009d4:  mov w3, #0x8320
0x4009e0:  movk w3, #0xedb8, lsl #16 ; w3 = 0xEDB88320 (reflected poly)
0x4009f0:  ... reflected fold: c = (c>>1) ^ ((c&1) ? poly : 0)
0x400a10:  mov x6, #0x0
0x400a70:  strb w1, [x20, x6] ; emit low byte of residual
0x400a78:  cmp x6, #0x20 ; 32 rounds

```

A reflected CRC-32 (polynomial 0xEDB88320 , no final XOR) is folded over the UID, then re-folded 32 times; each round emits the low byte. The key is a pure function of a **public** identifier.

**Threat model**

All four `system()` defects (B1, B2, B3, B5) are **local**: they require invoking the daemon or influencing a file/query it consumes. There is no network listener in the binary. The challenge is the **file/argument-to-root-exec primitive** and the **public-identifier key**, not a remote exploit.

**Grading Rubric (1-to-1 Mapping)**

| Criterion               | Points | Full Credit (Answer Key)        |
|-------------------------|--------|---------------------------------|
| [DOCUMENT] B1           | 5      | 0x400ae0 , run= compare, system |
| [DOCUMENT] B2           | 5      | "ping -c 1 %s" + system         |
| [DOCUMENT] B3           | 5      | "tar -xvzf %s -C /" + system    |
| [DOCUMENT] B4           | 5      | admin + empty password          |
| [DOCUMENT] B5           | 5      | system("/bin/sh")               |
| [DOCUMENT] B6           | 5      | reflected fold + 32 rounds      |
| [DOCUMENT] Threat model | -      | local, not remote               |

**Instructor Notes & Assembly**

- B1-B5 are proven live by `scripts/test_defects.py` ; B6 by `scripts/test_consistency.py` .
- Award full B3 credit for an end-to-end crafted archive even if the student does not name the exact function.

## Task 5: Break the Weak Key Schedule (20 points)

### Solution

#### The key schedule

```
/* ctf_crc32_le: reflected CRC-32, polynomial 0xEDB88320, no final XOR. */
uint32_t c = seed;
while (len--) c = fold(c, *data++);

/* ctf_weak_key */
uint32_t s = ctf_crc32_le(0, uid, strlen(uid)); /* initial hash over the UID */
for (int i = 0; i < 32; ++i) {
    s = ctf_crc32_le(s, uid, strlen(uid)); /* re-fold the UID */
    out[i] = s & 0xFF; /* emit the low byte */
}
```

So key = bytes 0..31 of (CRC32\_le^32( public\_UID )), low byte per round. The seed is 0 and there is **no** final inversion (this is the JFFS2-style `crc32_le`, not `zlib.crc32`).

#### Sample vector

For UID = SSAT-468547-FEEBD :

```
da506e04af00c6f40394d2cd2295bfc8682e8b9f9e9b844cea50c08d5f483141
```

Reproduce it exactly with the reference tool:

```
$ python3 scripts/weak_decrypt.py --uid SSAT-468547-FEEBD
da506e04af00c6f40394d2cd2295bfc8682e8b9f9e9b844cea50c08d5f483141
```

#### 200-Word Analysis (answer key)

A key derived from a public identifier is **obfuscation**, **not encryption**. The confidentiality of a cipher rests entirely on the secrecy of its key; if the key is a deterministic function of data an attacker already has (the device's public ID, a serial, a broadcast DID), then the attacker can recompute the key and decrypt - or forge - every message. The cipher may be strong, even AES, and the scheme still provides **zero confidentiality**, because the attacker never needs to break the cipher. This is the failure in the TELESCEEN exfiltration channel: the "sealed" payloads are sealed with a key the device hands out in the clear. Correcting it requires a **secret** key: either a pre-shared secret provisioned out of band, or a per-session key agreed with a real key exchange (X25519) and derived with a KDF (HKDF). Integrity must also be provided by an **AEAD** so forged beacons are rejected at the tag. Anything less is theatre.

#### Grading Rubric (1-to-1 Mapping)

| Criterion                 | Points | Full Credit (Answer Key)                                   |
|---------------------------|--------|------------------------------------------------------------|
| [DOCUMENT] Derivation     | 8      | poly 0xEDB88320 , seed 0, 32 low-byte rounds, no final XOR |
| [PATCH] Sample reproduced | 6      | da506e04...8341                                            |
| [DOCUMENT] 200 words      | 6      | public-ID key gives no confidentiality                     |

## Task 6: Demonstrate the Defects Locally (15 points)

### Solution

```
$ python3 scripts/test_defects.py
PASS B1 config-sourced root exec
PASS B2 CGI command injection
PASS B3 archive-to-root restore
PASS B4 default credentials accepted
PASS B4 wrong password rejected
PASS B5 debug root shell
FAILURES=0
```

```
$ python3 scripts/test_consistency.py
PASS python tool == published vector
PASS stripped binary == published vector
PASS python tool == stripped binary
python: da506e04af00c6f40394d2cd2295bfc8682e8b9f9e9b844cea50c08d5f483141
binary: da506e04af00c6f40394d2cd2295bfc8682e8b9f9e9b844cea50c08d5f483141
3/3 checks, 0 failures
```

| check              | what it proves                                                                |
|--------------------|-------------------------------------------------------------------------------|
| B1 pass            | a <code>run=</code> line in a config file creates a file as the node user     |
| B2 pass            | a <code>; touch</code> in the query creates a file                            |
| B3 pass            | a <code>tar</code> entry lands at <code>/tmp/d3</code> after the restore path |
| B4 accept + reject | <code>admin /empty</code> is accepted and a wrong password is not             |
| B5 pass            | the shell echoes input, so <code>/bin/sh</code> executed                      |
| consistency        | the binary's <code>key</code> output equals the Python tool and the vector    |

### Grading Rubric (1-to-1 Mapping)

| Criterion                                | Points | Full Credit (Answer Key) |
|------------------------------------------|--------|--------------------------|
| [PATCH] <code>test_defects.py</code>     | 6      | All checks PASS          |
| [PATCH] <code>test_consistency.py</code> | 5      | 3/3 checks, 0 failures   |
| [DOCUMENT] Interpretation                | 4      | One sentence per check   |

## Task 7: Write the Hardened Replacement (15 points)

### Solution

| defect | hardened behaviour                                                                                       |
|--------|----------------------------------------------------------------------------------------------------------|
| B1     | never source config as root; parse a fixed schema and dispatch an allow-list of typed actions in-process |

| defect | hardened behaviour                                                                                                                 |
|--------|------------------------------------------------------------------------------------------------------------------------------------|
| B2     | replace <code>system()</code> with <code>execvp("ping", {"ping", "-c", "1", query, NULL})</code> ; never build a command string    |
| B3     | reject absolute paths and any <code>..</code> entry; extract into a private staging dir, verify, then move with dropped privileges |
| B4     | remove the default <code>admin</code> /empty credential; require a provisioned secret and rate-limit                               |
| B5     | compile the debug shell out of production ( <code>#ifdef</code> ), and drop privileges before any shell                            |
| B6     | replace the public-ID KDF with X25519 ECDH + HKDF-SHA256 + AES-256-GCM                                                             |

AEAD design:

```
key source : X25519(device_ephemeral, collector_static) -> HKDF-SHA256 -> 32-byte key
cipher      : AES-256-GCM (ARMv8 crypto extensions)
nonce       : 96-bit, per message, never reused under one key
integrity    : 128-bit GCM tag; forged frames are rejected before parse
```

## Grading Rubric (1-to-1 Mapping)

| Criterion              | Points | Full Credit (Answer Key)                 |
|------------------------|--------|------------------------------------------|
| [DOCUMENT] Six fixes   | 9      | Table above                              |
| [DOCUMENT] AEAD design | 6      | X25519 + HKDF + AES-256-GCM, nonce + tag |

## Instructor Notes & Assembly

- Accept XChaCha20-Poly1305 in place of AES-256-GCM if nonce discipline is shown.
- The point of B6 is the **key source**, not the cipher; a student who keeps the public UID but wraps AES around it has missed the lesson.

## Task 8: The Defensive Playbook (bonus, 10 points)

### Solution

- Output/process detection:** alert on `system`-class execution from a daemon (`execsnoop` / `auditd`), on writes to the config file that feed `run=`, and on the `admin` /empty login in auth logs.
- Beacon detection:** periodicity/jitter analysis on outbound flows; payload-entropy analysis; flag the fixed collector endpoint; compare against a known-good baseline.
- Field hardening:** secure boot with a signed image; remove default credentials; replace the public-ID KDF with a secret-key AEAD; mount the rootfs read-only and require signed updates.

## Grading Rubric (1-to-1 Mapping)

| Criterion                           | Points | Full Credit (Answer Key)                                           |
|-------------------------------------|--------|--------------------------------------------------------------------|
| [DOCUMENT] Output/process detection | 4      | <code>system</code> sites, <code>run=</code> writes, default login |
| [DOCUMENT] Beacon detection         | 3      | Periodicity + entropy, concrete                                    |

| Criterion                         | Points | Full Credit (Answer Key)                   |
|-----------------------------------|--------|--------------------------------------------|
| <b>[DOCUMENT]</b> Field hardening | 3      | Secure boot, signed images, key management |

---

## Reference Material

- ARMv8-A Architecture Reference Manual (A64 instruction set)
- ELF64 / SysV AArch64 ABI; `.rela.plt` relocations
- Ghidra: <https://ghidra-sre.org/>
- NIST SP 800-38D (GCM); RFC 8439 (ChaCha20-Poly1305); RFC 7748 (X25519)
- `scripts/weak_decrypt.py`, `scripts/test_defects.py`, `scripts/test_consistency.py`